DevOps CI/CD Pipeline

End-to-End Project Documentation

GitHub → Jenkins → Maven → Apache Tomcat

Automated Build · Test · Deploy

Jenkins	Maven	GitHub	Tomcat

Author: Shibil Basith | Project: java-maven | Branch: main
Platform: Linux | GitHub: github.com/Shibil-Basith/java-maven

1. Introduction

What is CI/CD?

CI/CD stands for Continuous Integration and Continuous Deployment — a modern software development practice that automates the process of building, testing, and deploying applications. It forms the backbone of DevOps culture by removing manual handoffs and reducing time-to-production.

Continuous Integration (CI)	Continuous Deployment (CD)
Developers frequently merge code to a shared repository. Each merge triggers an automated build and test cycle, catching bugs early.	After a successful CI build, the application artifact is automatically deployed to a staging or production environment — no manual steps.

Why CI/CD Matters in Modern DevOps

- Reduces manual errors in build and deployment processes
- Provides fast feedback loops — developers know within minutes if a change breaks something
- Enables teams to release software frequently and reliably
- Increases developer confidence through automated testing and predictable deployments
- Bridges the gap between development and operations teams

Project Objective

This project demonstrates a fully functional, end-to-end CI/CD pipeline using industry-standard open-source tools. The pipeline automatically:

- Detects code changes pushed to a GitHub repository via Webhooks
- Triggers a Jenkins build job on the controller node
- Compiles and packages the Java application using Maven
- Transfers the build artifact (WAR file) to a remote agent node
- Deploys the WAR file to Apache Tomcat for immediate web access

> This entire workflow — from a git push to a live web app — completes automatically without any human intervention, illustrating real-world DevOps automation.

---

2. Architecture Overview

The pipeline follows a master-agent architecture where different stages of the pipeline run on different machines, separating concerns and mimicking a real production environment.

Architecture Components

Component	Description
Jenkins Controller	The main Jenkins server (built-in node). Orchestrates the pipeline, triggers builds, and communicates with the GitHub webhook. Runs the build stage using Maven.
Jenkins Agent (Slave)	A separate machine (or container) connected to the controller via SSH. Receives the build artifact and deploys it to Tomcat. Runs the deploy stage.
GitHub Repository	Remote version control system. Stores the Java source code and Maven project structure. Connected to Jenkins via a webhook for automatic trigger on push events.
Webhook	An HTTP callback mechanism configured in GitHub. On every git push to the main branch, GitHub sends a POST request to Jenkins, triggering the pipeline automatically.
Apache Tomcat	Java Servlet container running on the agent node. Serves the deployed web application and auto-deploys WAR files placed in its webapps/ directory.

Data Flow: GitHub to Deployment

1	**Code Push**	Developer pushes code to GitHub main branch
2	**Webhook Trigger**	GitHub sends HTTP POST to Jenkins webhook URL
3	**Jenkins Receives**	Jenkins controller detects the trigger and starts pipeline
4	**Source Checkout**	Jenkins clones the repository from GitHub
5	**Maven Build**	mvn clean package compiles Java code and creates WAR file
6	**Stash Artifact**	Jenkins stashes the WAR file internally for transfer
7	**Agent Pickup**	Deploy stage runs on slave agent; unstashes the WAR
8	**Rename & Copy**	WAR is renamed to ROOT.war and copied to Tomcat webapps/
9	**Tomcat Deploy**	Tomcat auto-detects and deploys the application
10	**App Live**	Application is accessible via http://<agent-ip>:8080/

3. Tools & Technologies Used

Tool	Purpose	Pipeline Role
OpenJDK	Java Development Kit — provides the Java runtime environment required by Jenkins, Maven, and the application itself	Runtime for all Java-based tools in the pipeline
Maven	Build automation tool for Java. Reads pom.xml to compile source, run tests, and package into a WAR file	Build stage on Jenkins controller
Git	Distributed version control system. Tracks code changes and enables collaboration	Source code management; used to clone repo in pipeline
GitHub	Cloud-hosted Git repository. Stores project source code and triggers Jenkins via webhooks on push events	Central code repository + webhook event source
Jenkins	Open-source automation server. Reads the Jenkinsfile pipeline, orchestrates build and deploy stages across nodes	Core CI/CD engine — coordinates the entire pipeline
Apache Tomcat	Java Servlet container that serves web applications packaged as WAR files. Auto-deploys apps in webapps/ directory	Application server on the agent — runs the final app

4. Environment Setup

4.1 Jenkins Controller Node Setup

Step 1: Install OpenJDK

```
sudo apt update
sudo apt install openjdk-17-jdk -y
java -version
```

Step 2: Install Maven

```
sudo apt install maven -y
```

```
mvn -version
```

Step 3: Install Git

```
sudo apt install git -y
git --version
```

Step 4: Install Jenkins

```
# Add Jenkins repository key
curl -fsSL https://pkg.jenkins.io/debian-stable/jenkins.io-2023.key | sudo tee \
  /usr/share/keyrings/jenkins-keyring.asc > /dev/null

# Add Jenkins repo to sources
echo deb [signed-by=/usr/share/keyrings/jenkins-keyring.asc] \
  https://pkg.jenkins.io/debian-stable binary/ | sudo tee \
  /etc/apt/sources.list.d/jenkins.list > /dev/null

sudo apt update
sudo apt install jenkins -y
sudo systemctl start jenkins
sudo systemctl enable jenkins
```

Step 5: Access Jenkins

```
# Jenkins runs on port 8080 by default
# Access via: http://<controller-ip>:8080

# Get initial admin password
sudo cat /var/lib/jenkins/secrets/initialAdminPassword
```

> After initial setup: Install suggested plugins, create an admin user, and configure the Jenkins URL in Manage Jenkins > Configure System.

4.2 Tomcat Agent Node Setup

Step 1: Install OpenJDK

```
sudo apt update
sudo apt install openjdk-17-jdk -y
```

Step 2: Install Apache Tomcat

```
# Create a dedicated user for Tomcat
sudo useradd -m -U -d /opt/tomcat -s /bin/false tomcat

# Download Tomcat (replace version as needed)
cd /tmp
wget https://dlcdn.apache.org/tomcat/tomcat-10/v10.1.x/bin/apache-tomcat-
10.1.x.tar.gz

# Extract and move to /opt/tomcat
sudo tar -xzf apache-tomcat-*.tar.gz -C /opt/tomcat --strip-components=1

# Set permissions
sudo chown -R tomcat: /opt/tomcat
sudo chmod +x /opt/tomcat/bin/*.sh
```

Step 3: Configure Tomcat to Run as a Service

```
sudo nano /etc/systemd/system/tomcat.service

[Unit]
Description=Apache Tomcat Web Application Server
After=network.target

[Service]
Type=forking
User=tomcat
Group=tomcat
Environment="JAVA_HOME=/usr/lib/jvm/java-17-openjdk-amd64"
Environment="CATALINA_HOME=/opt/tomcat"
ExecStart=/opt/tomcat/bin/startup.sh
ExecStop=/opt/tomcat/bin/shutdown.sh

[Install]
WantedBy=multi-user.target
```

```
sudo systemctl daemon-reload
sudo systemctl start tomcat
sudo systemctl enable tomcat
```

Step 4: Set webapps Permissions for Jenkins

```
# Allow Jenkins to copy files to Tomcat webapps
sudo chmod 777 /opt/tomcat/webapps/
```

---

5. Jenkins Agent (Master-Agent) Setup

What is Master-Agent Architecture?

Jenkins follows a master-agent (controller-agent) model to distribute workloads. The controller manages job scheduling, user interface, and pipeline orchestration, while agents execute the actual build and deployment tasks. This allows:

- Running stages on different machines (e.g., build on controller, deploy on target server)
- Parallel execution of stages on multiple agents simultaneously
- Isolation of environments — keeping build tools separate from deployment environments
- Scalability — add more agents as workload increases

Connecting the Agent via SSH

Step 1: Generate SSH Key on Jenkins Controller

```
# Run as jenkins user
sudo -u jenkins ssh-keygen -t rsa -b 4096 -C 'jenkins-agent'
# Key stored at: /var/lib/jenkins/.ssh/id_rsa (private)
#                /var/lib/jenkins/.ssh/id_rsa.pub (public)
```

Step 2: Copy Public Key to Agent Machine

```
# On the agent machine, add public key to authorized_keys
mkdir -p ~/.ssh
echo '<paste-public-key-content>' >> ~/.ssh/authorized_keys
chmod 700 ~/.ssh && chmod 600 ~/.ssh/authorized_keys
```

Step 3: Add SSH Credentials in Jenkins

- Go to: Manage Jenkins > Credentials > Global > Add Credentials
- Kind: SSH Username with private key
- ID: slave-ssh-key (or any name)
- Username: ubuntu (or the agent machine user)
- Private Key: Paste the content of /var/lib/jenkins/.ssh/id_rsa

Step 4: Register the Agent Node

- Go to: Manage Jenkins > Nodes > New Node
- Node name: slave
- Type: Permanent Agent
- Remote root directory: /home/ubuntu/jenkins-agent
- Labels: slave (this is used in the Jenkinsfile as agent { label 'slave' })
- Launch method: Launch agents via SSH
- Host: <agent-machine-IP>
- Credentials: Select the SSH key created above

> After saving, Jenkins will SSH into the agent and launch the agent process. You will see it go online in the Nodes section. This label 'slave' is what the Jenkinsfile uses to route the deploy stage to this machine.

---

6. GitHub Repository Setup

Repository Structure

```
java-maven/
├── src/
│   └── main/
│       ├── java/
│       │   └── com/example/App.java
│       └── webapp/
│           └── WEB-INF/
│               └── web.xml
├── pom.xml
└── Jenkinsfile
```

Key Files Explained

- **pom.xml:** Maven project descriptor. Defines the project group, artifact ID, packaging type (war), dependencies, and build plugins.
- **Jenkinsfile:** Defines the complete CI/CD pipeline as code. Jenkins reads this file from the repo to know what to do and where.
- **src/:** Contains all Java source code and web application resources (JSP, HTML, web.xml).

Pushing Code to GitHub

```
# Initialize local repo
git init
git add .
git commit -m 'Initial commit: Java Maven web app'

# Link to remote GitHub repository
git remote add origin https://github.com/Shibil-Basith/java-maven.git

# Push to main branch
git push -u origin main
```

---

7. GitHub Webhook Configuration

What is a Webhook?

A webhook is an HTTP callback — GitHub sends an automatic HTTP POST request to a specified URL whenever a specific event occurs in the repository (such as a push). This is the trigger mechanism that connects GitHub to Jenkins.

Without webhooks, Jenkins would have to poll GitHub repeatedly to check for new commits — an inefficient approach. With webhooks, the integration is event-driven and near-instant.

Configuring the Webhook in GitHub

1	**Open Repo Settings**	Go to your GitHub repository > Settings > Webhooks > Add webhook
2	**Set Payload URL**	Enter: http://<jenkins-controller-ip>:8080/github-webhook/

3	**Content Type**	Select: application/json
4	**Choose Events**	Select 'Just the push event' to trigger on every git push
5	**Save**	Click 'Add webhook'. GitHub will send a ping to verify connectivity

Configuring Jenkins to Accept Webhooks

- In your Jenkins job: Check 'GitHub hook trigger for GITScm polling' under Build Triggers
- In Manage Jenkins > Configure System: Set the correct Jenkins URL (used in webhook response)

Common Error: 403 Forbidden

> Problem: GitHub webhook receives a 403 Forbidden response from Jenkins. This is caused by Jenkins CSRF protection blocking external POST requests.

Solution

- Install the plugin: GitHub Integration Plugin (or GitHub plugin)
- This plugin properly handles GitHub webhook requests without CSRF conflicts
- Alternatively: In older Jenkins versions, navigate to Manage Jenkins > Configure Global Security and adjust CSRF settings — but using the plugin is the recommended approach

---

8. Jenkins Pipeline (CI/CD)

Complete Jenkinsfile

```
pipeline {
    agent none
    stages {
        stage("build") {
            agent {
                label 'built-in'
            }
            steps {
```

```
                git branch: 'main', url: 'https://github.com/Shibil-
Basith/java-maven.git'
                sh "mvn clean package"
                stash name: 'build', includes: 'target/*.war'
            }
        }
        stage('deploy') {
            agent {
                label 'slave'
            }
            steps {
                unstash 'build'
                sh "rm -rf target/ROOT.war"
                sh "mv target/*.war target/ROOT.war"
                sh "cp target/ROOT.war /opt/tomcat/webapps/"
            }
        }
    }
}
```

Pipeline Breakdown

agent none

Declaring agent none at the top means no global agent is assigned. Each stage independently declares which node it runs on. This is the correct approach for multi-node pipelines.

Stage: build — runs on 'built-in' (Controller)

- **git branch: 'main':** Clones the java-maven repository from GitHub, checking out the main branch
- **mvn clean package:** 'clean' deletes previous build output. 'package' compiles Java source and creates a WAR file inside the target/ directory
- **stash name: 'build':** Jenkins saves the WAR file(s) matching target/*.war into an internal storage called 'build'. This file is now available for any other node to retrieve

Stage: deploy — runs on 'slave' (Agent/Tomcat server)

- **unstash 'build':** Retrieves the previously stashed WAR file from Jenkins internal storage and recreates it on the agent machine
- **rm -rf target/ROOT.war:** Removes any previously renamed ROOT.war to avoid conflicts

- **mv target/*.war target/ROOT.war:** Renames the artifact to ROOT.war. Tomcat deploys ROOT.war to the root context path (/), making the app accessible at http://ip:8080/ directly
- **cp target/ROOT.war /opt/tomcat/webapps/:** Copies ROOT.war to Tomcat's deployment directory. Tomcat monitors this folder and auto-deploys any new WAR file

Why stash/unstash Instead of SCP?

stash / unstash	SCP (manual file transfer)
✓ Native Jenkins feature — no extra configuration	✗ Requires SSH setup, keys, and known hosts
✓ Works with any agent regardless of network topology	✗ Agents must be network-reachable from controller
✓ Handled by Jenkins internal channels	✗ Requires manual scripting
✓ Clean and portable across environments	✗ Tightly coupled to IP addresses / paths

---

9. Deployment Process

How WAR Deployment Works

A WAR (Web Application Archive) file is a standard Java web application package. Apache Tomcat has a built-in auto-deployment mechanism — any WAR file placed in the webapps/ directory is automatically extracted and deployed as a web application.

Deployment Steps in Detail

- **WAR Generation:** Maven's package lifecycle phase compiles all Java source files, runs tests, and packages everything (classes, resources, WEB-INF/) into a single .war archive inside the target/ directory.

- **Renaming to ROOT.war:** The generated WAR is named after the Maven artifactId (e.g., java-maven-1.0.war). By renaming it to ROOT.war, we instruct Tomcat to deploy it at the root context path. This means the app is accessible at http://ip:8080/ instead of http://ip:8080/java-maven-1.0/

- **Copy to webapps/:** Once ROOT.war lands in /opt/tomcat/webapps/, Tomcat's deployer thread detects the new file, extracts it into a ROOT/ directory, and makes the servlet available immediately. No restart required.

```
# Pipeline deploy steps
```

```
sh "rm -rf target/ROOT.war"          # Clean previous rename
sh "mv target/*.war target/ROOT.war"  # Rename to ROOT context
sh "cp target/ROOT.war /opt/tomcat/webapps/"  # Deploy
```

---

10. Accessing the Application

Application URL

> http://<agent-node-ip>:8080/ Because the WAR is named ROOT.war, Tomcat serves it at the root context path — no sub-path needed.

What Happens When You Access the URL

- **Browser → Tomcat:** The HTTP request reaches Tomcat on port 8080 of the agent machine
- **Root Context Mapping:** Tomcat maps the / path to the ROOT web application (extracted from ROOT.war)
- **Servlet Dispatch:** The request is routed to the configured Servlet/JSP defined in WEB-INF/web.xml
- **Response:** The Java web application generates and returns the HTTP response

Verifying Successful Deployment

```
# Check Tomcat webapps directory — ROOT/ folder should exist
ls /opt/tomcat/webapps/
# Expected output: ROOT  ROOT.war  (and possibly other default apps)

# Check Tomcat logs for deployment status
tail -f /opt/tomcat/logs/catalina.out
```

---

11. Challenges Faced & Solutions

Problem	Root Cause	Solution

mvn: command not found	Maven was not installed on the Jenkins controller node or not on the PATH	Install Maven: sudo apt install maven -y. Verify with mvn -version
GitHub 403 on git clone	Repository URL was wrong or repo was private without credentials	Use correct public repo URL or configure GitHub credentials in Jenkins
Webhook 403 Forbidden	Jenkins CSRF protection rejecting external POST from GitHub	Install the GitHub Integration Plugin; it handles webhook auth correctly
Multiple WAR files conflict	Previous builds left old WAR files causing ambiguous mv target/*.war	Added rm -rf target/ROOT.war before the mv command to always start clean
Tomcat not auto-deploying	Incorrect permissions on webapps/ directory	chmod 777 /opt/tomcat/webapps/ to allow Jenkins agent to write files
Agent offline in Jenkins	SSH key mismatch or agent machine firewall blocking port 22	Verified public key in authorized_keys; ensured port 22 open on agent

---

12. Security Considerations

SSH Key Authentication

- **Private key stored securely:** The Jenkins controller stores the private SSH key in its credential store (encrypted), never as a plain text file accessible to users.
- **No password authentication:** SSH is configured to use key-based auth only. This eliminates the risk of brute-force password attacks on the agent.
- **Principle of least privilege:** The Jenkins agent user only has permissions to the directories it needs (webapps/, jenkins-agent/).

Webhook Security

- **Network exposure:** The Jenkins webhook URL must be publicly accessible for GitHub to reach it. Limit this by placing Jenkins behind a reverse proxy (NGINX) with HTTPS.
- **GitHub webhook secrets:** Configure a secret token in the GitHub webhook settings and verify it in Jenkins to ensure only GitHub can trigger builds.
- **HTTPS for Jenkins:** In production, always serve Jenkins over HTTPS using a valid TLS certificate.

Additional Hardening

- Disable the Jenkins built-in node for builds in production — use only dedicated agents
- Enable Jenkins Role-Based Access Control (RBAC) to limit what each user can do

- Regularly rotate SSH keys and audit authorized_keys on agent machines
- Use a secrets manager (HashiCorp Vault, AWS Secrets Manager) for sensitive credentials

---

13. Improvements & Future Scope

Enhancement	Description & Benefit
Docker Integration	Package the application into a Docker image during the build stage instead of a WAR file. Deploy containers instead of Tomcat instances — enables consistent environments and easy rollbacks.
Kubernetes Deployment	Use Helm charts to deploy containers to a Kubernetes cluster. Adds auto-scaling, self-healing, and zero-downtime rolling updates.
Nexus Artifact Repository	Store build artifacts (WAR/JAR) in Sonatype Nexus or JFrog Artifactory. Enables artifact versioning, audit trail, and reuse across environments.
SonarQube Integration	Add a code quality analysis stage to the pipeline using SonarQube. Catches code smells, security vulnerabilities, and test coverage gaps before deployment.
Blue-Green Deployment	Maintain two identical production environments (blue and green). Route traffic to the new version only after it passes health checks — enables zero-downtime deployments.
Automated Testing	Add a dedicated test stage (unit, integration, smoke tests) between build and deploy. Only deploy if all tests pass.
Slack/Email Notifications	Configure Jenkins to send build status notifications to Slack or email, keeping the team informed of failures instantly.

---

14. Key Learnings

Technical Skills Gained

- **Jenkins Pipeline as Code:** Writing Jenkinsfile using declarative pipeline syntax. Understanding how stages, agents, and steps compose an automated workflow.
- **Master-Agent Architecture:** Setting up and connecting Jenkins agents via SSH. Understanding how to route pipeline stages to specific machines using labels.
- **Maven Build Lifecycle:** Understanding how clean, compile, test, and package phases work. How Maven's pom.xml drives the entire build process.
- **Artifact Management:** Using Jenkins stash/unstash to reliably transfer build artifacts between pipeline stages running on different nodes.

- **Tomcat Deployment:** How WAR auto-deployment works, the significance of ROOT.war for the root context, and Tomcat's webapps/ directory behavior.
- **Webhook-Driven Automation:** Configuring event-driven CI/CD triggers using GitHub webhooks. Understanding the request/response flow between GitHub and Jenkins.

DevOps Principles Reinforced

- Infrastructure as Code — pipeline defined in Jenkinsfile, version-controlled with the application
- Automation over manual processes — zero human steps from push to deployment
- Fast feedback — failed builds are reported immediately, reducing the cost of fixing bugs
- Separation of concerns — build environment is separate from deployment environment

Project Summary

What was built	Technologies mastered
A fully automated CI/CD pipeline that builds a Java Maven web application and deploys it to Apache Tomcat on every GitHub push — end to end, hands-free.	Jenkins (pipelines, agents, credentials) · Maven (build lifecycle) · GitHub (webhooks, version control) · Apache Tomcat (WAR deployment) · Linux (server config, SSH, permissions)